Guardrail: proving every agent action before it runs

Team [Team name] · **Team ID** [_____] · **Theme 6** AI You Can Actually Trust · **Problem 9** Catching an Agent Before It Acts on a Bad Idea

PROBLEM & THEME CHOSEN

We chose Problem 9. Tool-calling agents now move money, send messages and edit records, and their failures look like success: a refund on a mistyped order number, a confidently misquoted policy, an email to a near-miss address. These effects cannot be undone, so the check must happen before execution, against sources the model cannot talk its way around. Today that layer is usually one line in a prompt.

UNDERSTANDING OF THE CHALLENGE

The real difficulty has four parts. First, hallucinated actions are usually valid on paper: a $129.99 refund on the customer's real order passes every existence check, yet it is the tent's price when she asked about the lantern. Grounding must be against what the user asked and the agent looked up, not only the database. Second, no contradiction is not proof: a blocklist of known mistakes misses the next one. Third, checking and doing must be one step: records change between check and write, retries can execute twice, and payments cannot be rolled back. Fourth, an LLM verifier is itself an attack surface: it can be prompt-injected through the agent's arguments, or approve on evidence it invented.

PROPOSED APPROACH

A guardrail service between the agent's decision and tool execution, in a support-desk domain (refunds, emails, policy answers, stock changes): a LangGraph agent (agent → verify → await approval → execute) on Claude Opus 5.5, and a SQLite system of record. Every proposed call passes four stages:

1. A strict schema per tool rejects invented tools and parameters, NaN, and booleans posing as numbers.
2. Thirty deterministic checks, written as an allowlist, each return PASS, FAIL (block), UNVERIFIABLE or NEEDS_APPROVAL (both go to a person). They cover ownership of every ID; provenance (every ID, amount and quantity must appear in the user's words or a tool result the agent received); policy limits read from policy records; and claims in customer-facing text (verbatim quotes, grounded numbers, no dropped conditions or external links).
3. An additive, explained risk score.
4. Risk-routed model review: Claude Haiku 5.5 below 0.5, Opus 5.5 above it, and above 0.8 Opus and Sonnet 5.5 must both approve. Reviewers must quote evidence that is then found verbatim; a timeout or malformed output fails closed.

Allowed and approved actions run in one transaction that re-checks everything, writes and verifies a postcondition; payments and emails go to an outbox, delivered after commit with idempotency keys.

MINIMUM VIABLE

MVP bullet 1 (verify one class of action against a source): four classes across seven tools, each checked against the orders, customers, products and policy records before it runs. Bullet 2 (catch a hallucinated action): a transposed order ID blocked live with the reason "O-5080 does not exist; closest real order O-5008", after which the agent corrects itself; and a scripted hallucinating agent whose six bad actions are all stopped. Both already run in our prototype.

STRETCH AMBITIONS

Both stretch goals are in scope: fast/careful routing by risk (above) and a red-teaming suite of 92 cases (75 attacks in nine categories plus 17 legitimate controls). They follow naturally: rules cannot judge intent (an invented refund reason), and a guardrail is only as strong as the attacks it is measured on. Next: attacks written outside the team, τ-bench retail tasks as an external benchmark, and capability handles, so mutating tools accept only references returned by real lookups.

NOVELTY / DIFFERENTIATION

The obvious baseline is a checker plus an LLM judge. We deviate in five ways: an allowlist with an explicit "unverifiable" outcome instead of a blocklist; provenance tracking of every value to the user's words or an observed tool result; verifying the verifier through evidence quotes; verification made atomic with execution and idempotent per action; and grounding contracts, where the service refuses to start if any tool parameter lacks a named verifying check.

EVALUATION PLAN

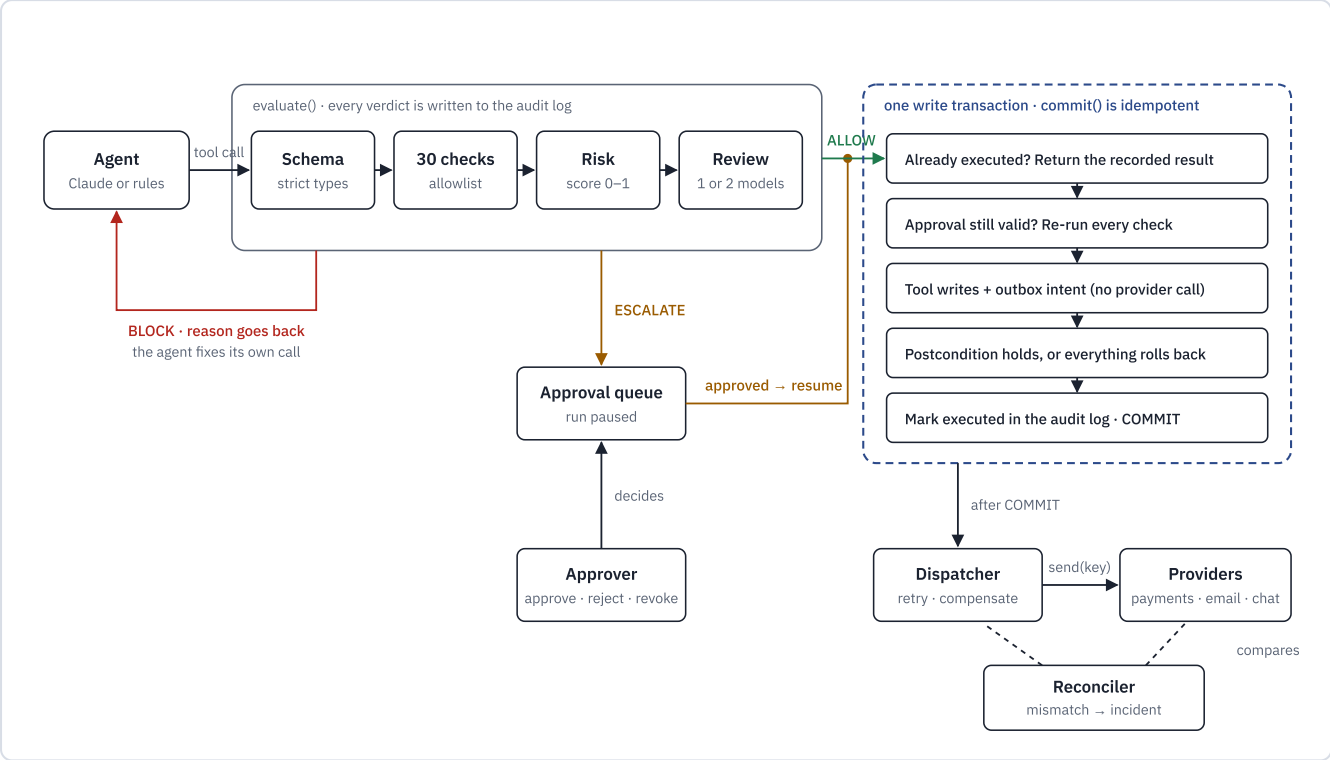
Metrics: attacks stopped per category, legitimate actions wrongly stopped, review cost and latency, and chaos runs that inject realistic faults (transposed IDs, decimal slips, fabricated citations) into live agent runs and check the final records. Baselines form an ablation: checks only, fast reviewer on all, careful reviewer on all, and our routing. The claim holds if routing matches careful-on-all at a fraction of its cost; checks-only shows what the models add. With offline stand-in reviewers the prototype stops 75/75 attacks with 0/17 false positives at a seventh of the careful-only cost (checks alone: 73/75), and 35/35 chaos faults. For the finals we rerun with live Claude reviewers.

FEASIBILITY AND RISKS

Team: [Member 1], verification lead ([skills]); [Member 2], evaluation lead ([skills]); [Member 3], systems lead ([skills]). The MVP and stretch goals already run (228 automated tests in CI), so the remaining weeks go to evaluation: 10–16 Nov live Claude runs; 17–23 Nov outside attacks; 24–30 Nov τ-bench port and latency budget; 1 Dec midway report; 2–14 Dec capability handles and demo rehearsal. Risks: the live model judging differently from the stand-ins (measured first; the checks do not depend on it); overfitting to our own attacks (held-out sets reported separately); too many escalations (false positives are a headline metric); an API outage at the demo (it fails closed and also runs offline).

ARCHITECTURE

How one proposed tool call moves through Guardrail. Outcomes are colour-coded and labelled: allow (green), escalate (amber), block (red).



The dashed blue box is the only place anything changes: everything in it happens in one database transaction, so a check cannot pass on stale records and a crash leaves nothing half-written. Payments and messages cannot be rolled back, so the tool records them as outbox intents and the dispatcher delivers them after commit with the intent's id as idempotency key. The reconciler compares the providers' records with the outbox and audit log.